Computer Programming 1

Computers – Structure and Behavior

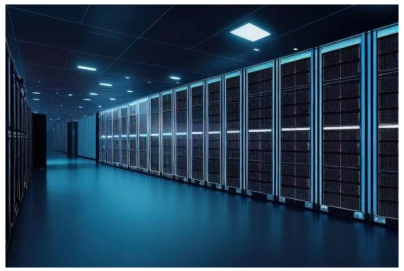
Computers and Operating Systems



- C++ is a programming language used for a wide variety of systems, from the small to the large
- Today we will have a look at:
  - some terminology (both for hardware and software)
  - delve into the functioning model of a computer (Von Neumann architecture)
  - describe the operating environment (OS and shell)

Types of “computers”

- Supercomputer
  - A powerful computer that can process large amounts of data and do a great amount of computation very quickly.
- Mainframe
  - A high-performance computer used for large information processing jobs.
  - Large size and high power
  - Shared among many users
- Personal computer (Desktop)
  - Equipped with monitor and keyboard separate from the central unit
  - Equipped with various mass storage devices inside or outside the central unit
- Laptop
  - Portable version of the personal computer
- Handheld computer (Smartphone)
  - Limited size
  - A wireless touchscreen PC that is slightly smaller and weighs than normal PCs.
- Embedded systems (e.g., smartwatches)





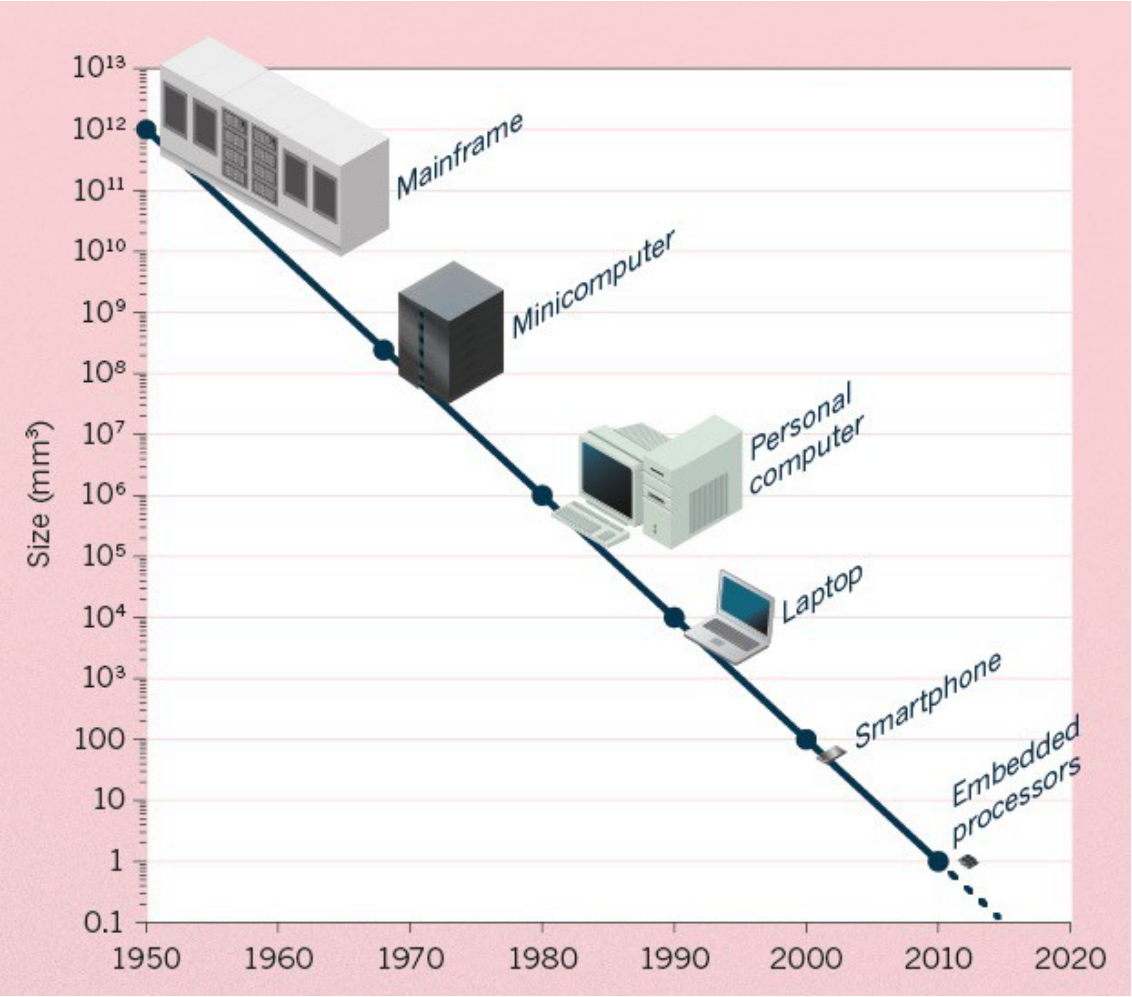




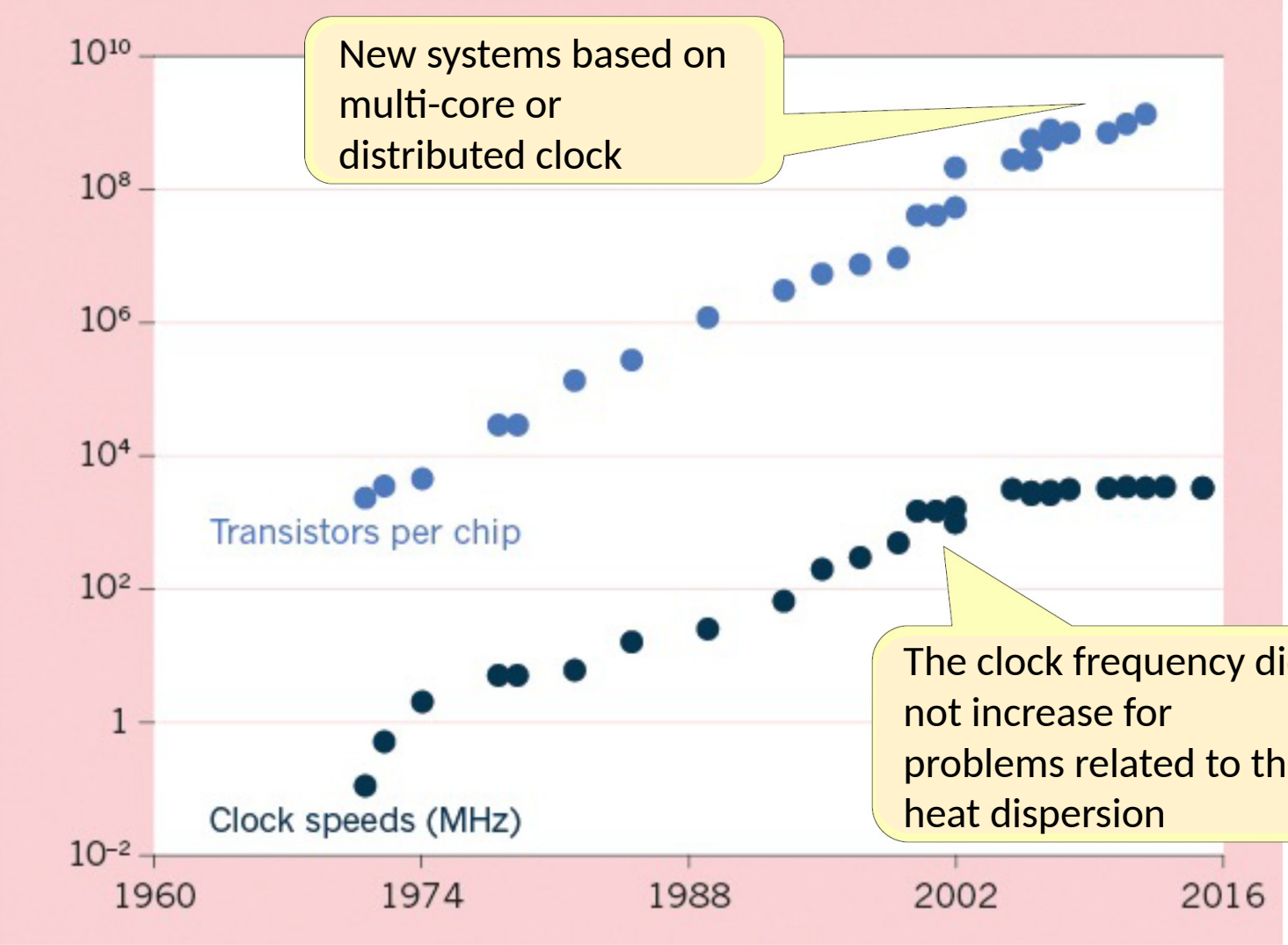




Their size...


Size (mm³)
10¹³
10¹²
10¹¹
10¹⁰
10⁹
10⁸
10⁷
10⁶
10⁵
10⁴
10³
100
10
1
0.1
1950
1960
1970
1980
1990
2000
2010
2020
Mainframe
Minicomputer
Personal computer
Laptop
Smartphone
Embedded processors

Transistors & speed: Moore's law

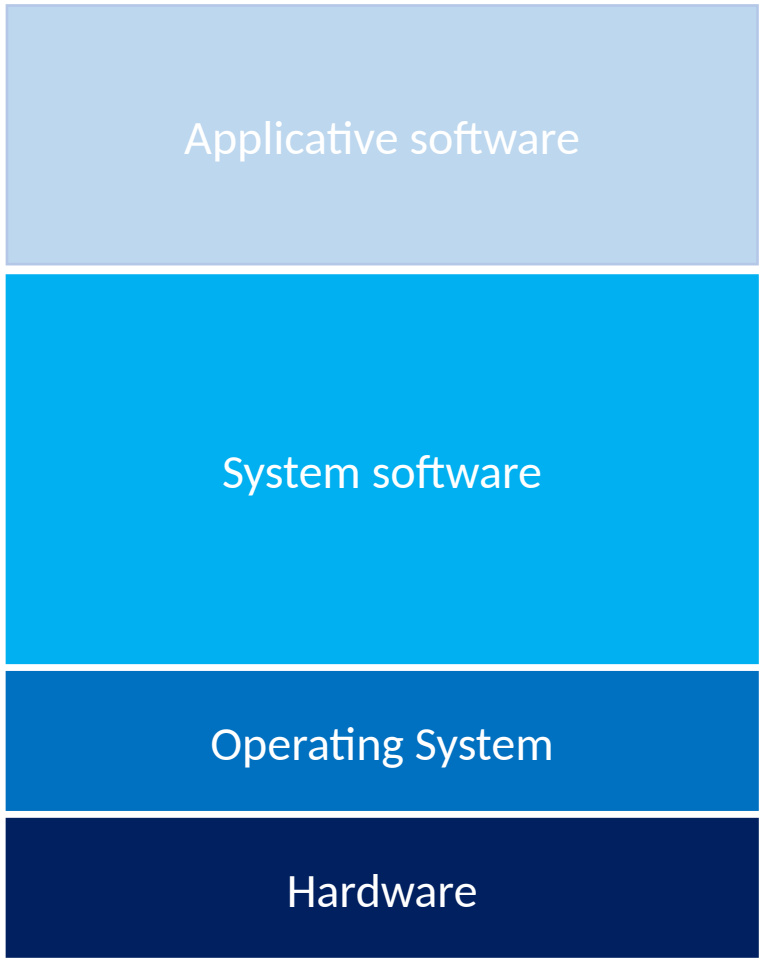


A transistor is a semiconductor device used in electronic devices

"The number of transistors that can be integrated on a circuit roughly doubles every 2 years"
G.Moore, 1965

Is the Moore's law at the end ? … new technologies are needed

SW organization in a Computer



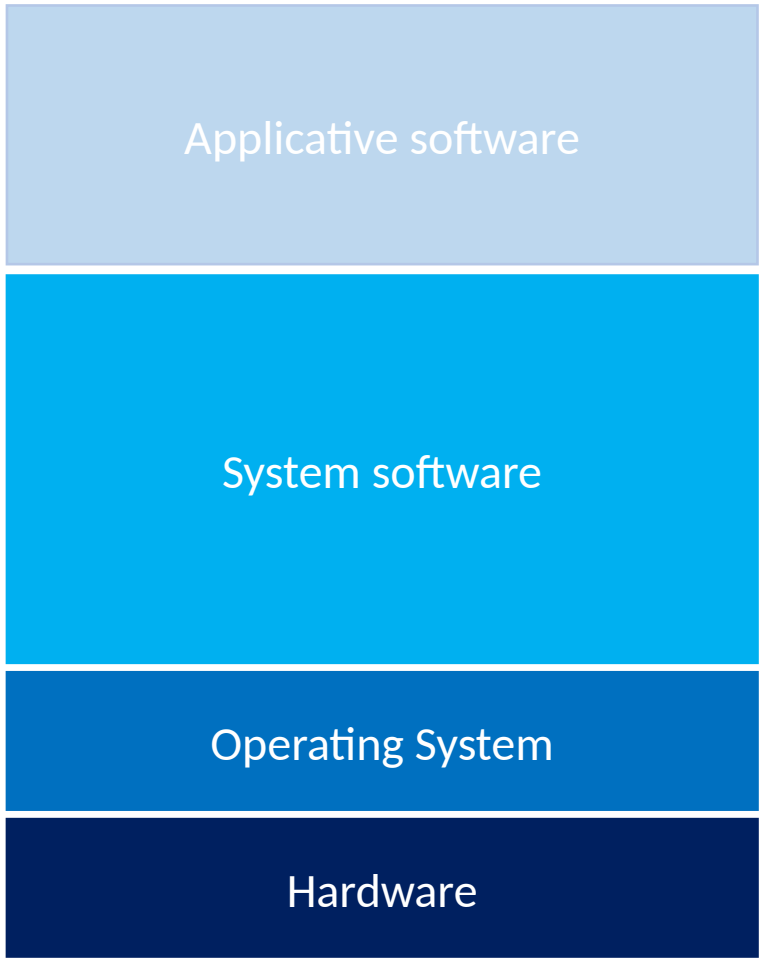
Individual Productivity software
(e.g., spreadsheets, mail readers, browsers)

Programming environment
(e.g., Compiler, debugger)

(e.g., Windows, Linux, Mac OS)

Physical computer's components

SW organization in a Computer



Individual Productivity software (e.g., spreadsheets, mail readers)

Programming environment (e.g., Compiler, debugger)

(e.g., Windows, Linux, Mac OS)

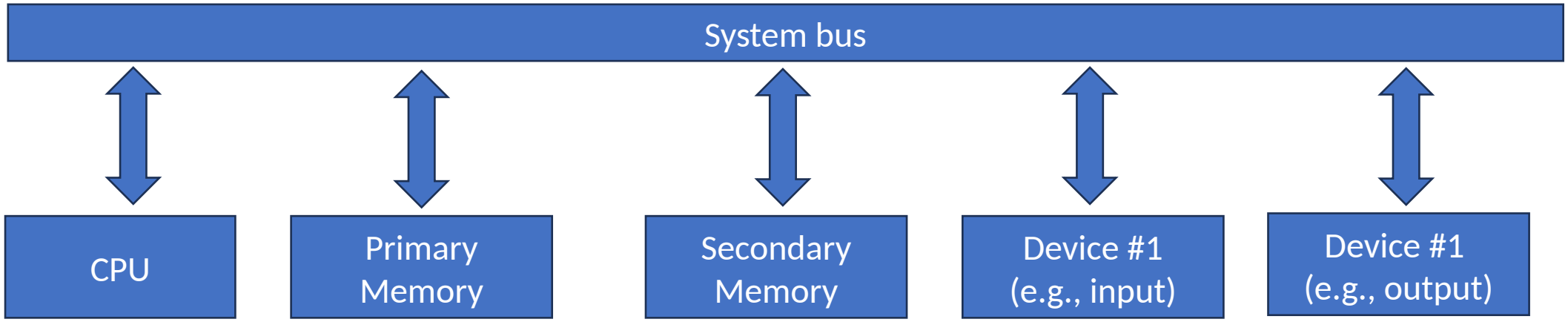
Physical computer's components

Computer Hardware architecture (1)

- The conventional hardware architecture is named “Von Neuman Architecture”, after the Hungarian mathematician who first proposed the architecture of a computer (~1945)
- Some remarkable aspects:
  - The architecture is universal, allowing the same machine to be programmed and used for different purposes (*)
  - All modern computers, from mainframes to embedded systems, today are based on the Von Neumann architecture

(*) this makes a Von Neumann computer different from, e.g., a calculator – even if built with the same hardware components - can perform only one function/program

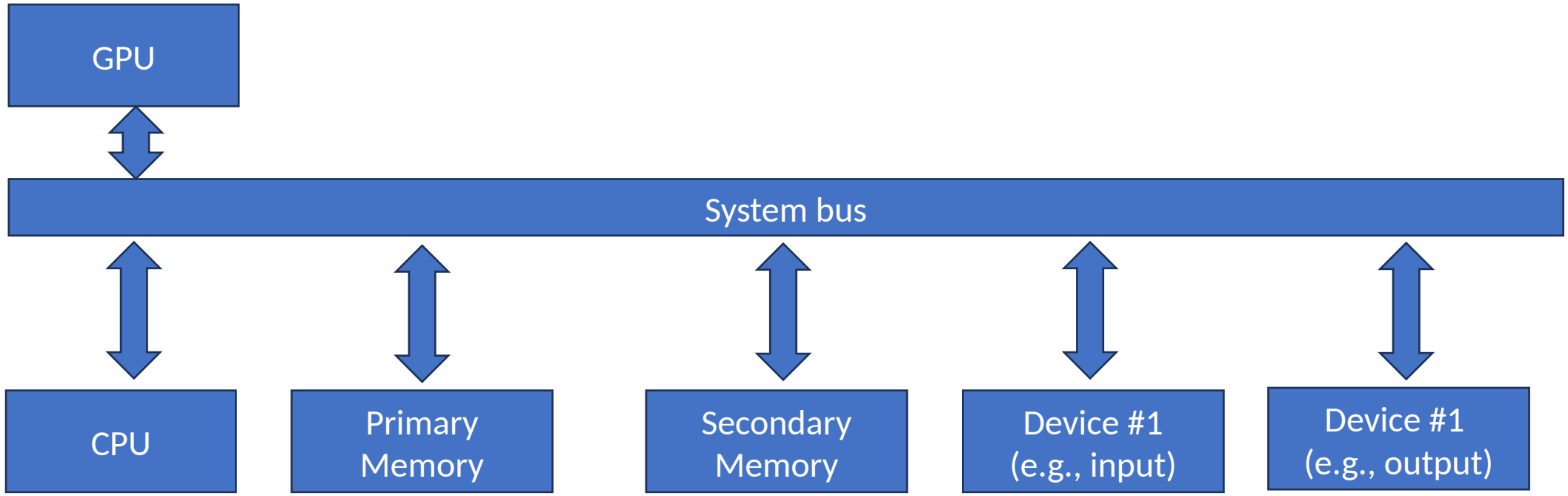
Computer Hardware architecture (2)



- System Bus: interconnects other components, allowing data exchange
- CPU : performs processing, "executing programs"
- Primary memory: used to store data and instructions
- Secondary memory: used to store large amounts of data and programs, persistently (i.e., non-volatile, as opposed to RAM)
- Device units: various types, mass storage, monitors, keyboards, network cards, …

Von Neumann Architecture (3)

- In modern architectures:
  - The CPU is often supported by a GPU (Graphical Processor Unit), a processor specialized in the execution of mathematical operations
  - CPUs are organized in multiple independent cores, each core being a CPU and being able to execute instructions in parallel with the other cores
  - CPUs have local memories which they can use to store data which they need to access frequently



How the information is represented

- Binary representation of the information: BIT 0 and/or 1

Being structurally based on bi-stable devices (e.g., current ON, current OFF), the computer can only operate on sequences of binary symbols

The two conventionally used symbols are 0 and 1

A “BIT” (Bynary digIT) hence is the elementary unit of information

- Instructions and data are represented in the computer by long sequences of binary digits
- The BYTE is a sequence of 8 BITs

Binary representation of characters

Everything is a binary number, but we can give numbers different meanings, according to the context in which they appear.

The representation of characters is done by reserving a byte for each character and checking on a table (extended ASCII) that gives the correspondence between the sequence and the character.

For instance, the character 'a' is represented by:

$(97)_{10} = (01100001)_2$

8 bits = 1 byte

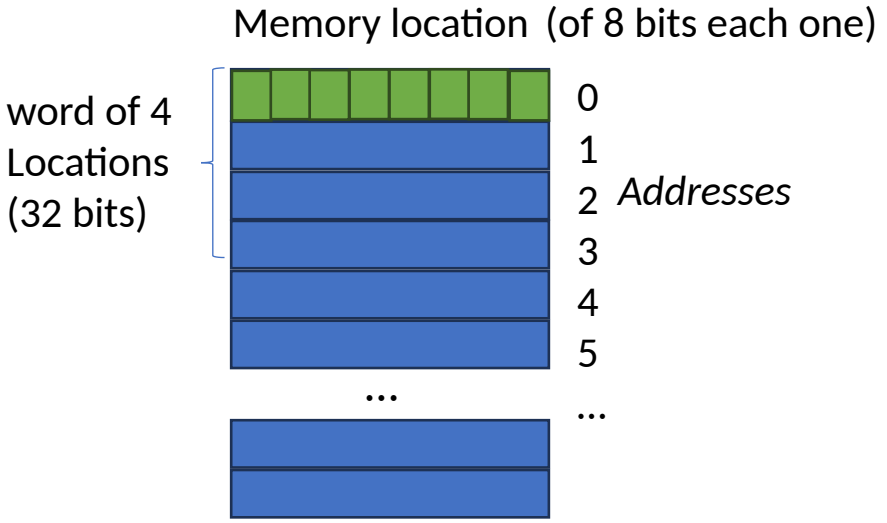
Name	Approximation	Value	
KILOBYTE	KB	$2^{10} = 1024$ byte	~10^3 byte
MEGABYTE	MB	$2^{20} = 1024^2 = 1024$ KB	~10^6 byte
GIGABYTE	GB	$2^{30} = 1024^3 = 1024$ MB	~10^9 byte
TERABYTE	TB	$2^{40} = 1024^4 = 1024$ GB	~10^{12} byte
PETABYTE	PB	$2^{50} = 1024^5 = 1024$ TB	~10^{15} byte
EXABYTE	EB	$2^{60} = 1024^6 = 1024$ PB	~10^{18} byte

Memory (1)

The primary/central memory stores data and programs on which the computer operates

- Conceptually, the memory is composed of a sequence of cells, storing "**words**"
- A **word** is composed of a number of bits that depends on the machine. Examples => Pentium: 32 bits, PowerPC: 32 bits, UltraSPARC: 64
- Each cell can be accessed directly (specifying its address) (Compare with a magnetic tape, for instance)

- Since everything is stored as binary numbers, a computer distinguishes data from program by storing them in different areas of the memory (or using special bits)



Memory (2)

RAM (Random Access Memory)

The primary memory is usually made up of a RAM

It is a memory whose access time is independent of the address of the word to which you want to access

- It is a volatile memory, i.e., the contents are lost when the system ceases to power
- In DRAM (Dynamic RAM) the content must even be "refreshed" periodically, thus allowing higher densities than SRAM (static RAM)

ROM (Read-Only Memory)

They have general characteristics similar to RAM

Very fast access

These are permanent memories

- Typically used to store data and programs that are needed at the time of switching on the computer, before loading the O.S.
  - E.g. BIOS (Basic Input-Output System)

Secondary Memory (Mass storage)

- Mass storage refers to a permanent storage device capable of holding large amounts of data.

The trade-off between RAM, ROM, and mass storage, **costs, speed, and flexibility/durability.**

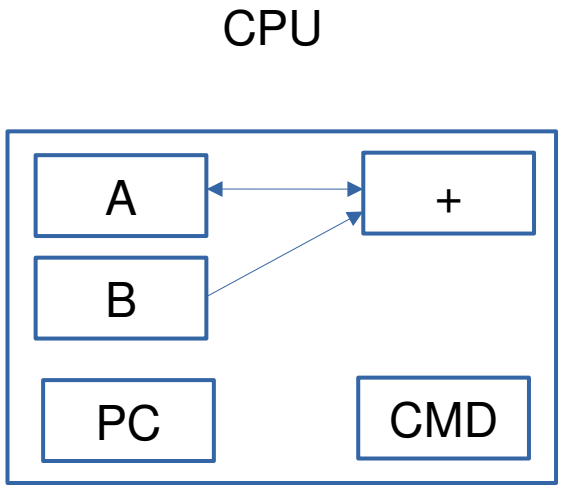
RAM is volatile

ROM is permanent, but can be written to only once

Mass storage is permanent (more or less), can be written multiple times, is (was) cheaper than RAM, but it is slower than RAM

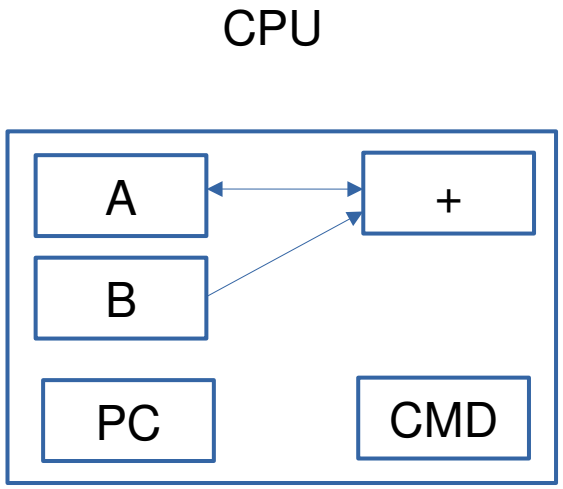
Simplified Execution Model: CPU

- The CPU is the “brain”
- CPUs can perform simple operations
- A typical set of operations a CPU can execute are:
  - READ/WRITE from memory (and other devices)
  - Perform mathematical operations
  - Perform logical operations
  - Control the flow of instructions (which instruction is next)



Simplified Execution Model: Assembly

- The set of instructions understood by a processor is limited and simple
- Each instruction can be represented by a binary number
- Each instruction usually also has a human-readable representation (assembly)
- A program in assembly" might look like this

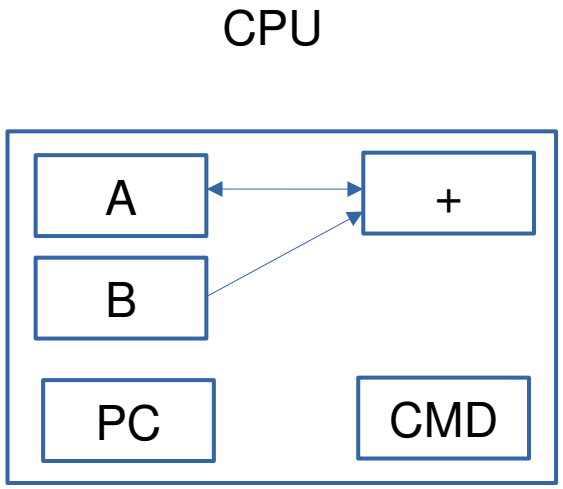



```
; show "0x" as a prefix
  ld a, '0'
  out(1),a
  ld a, 'x'
  out(1),a
```

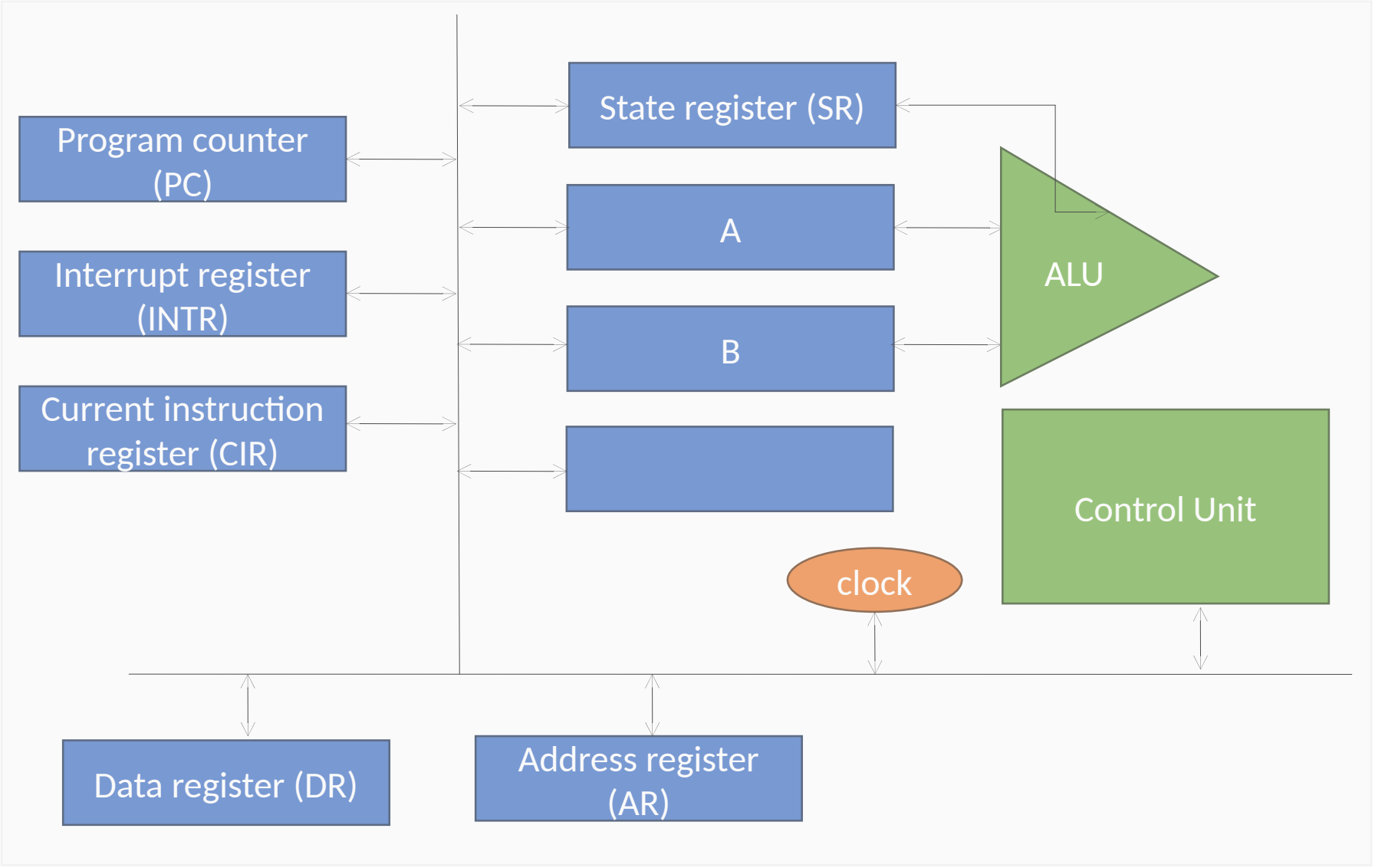
Assembly Language

- When you compile a program, C++ instructions are translated from C++ to assembly language

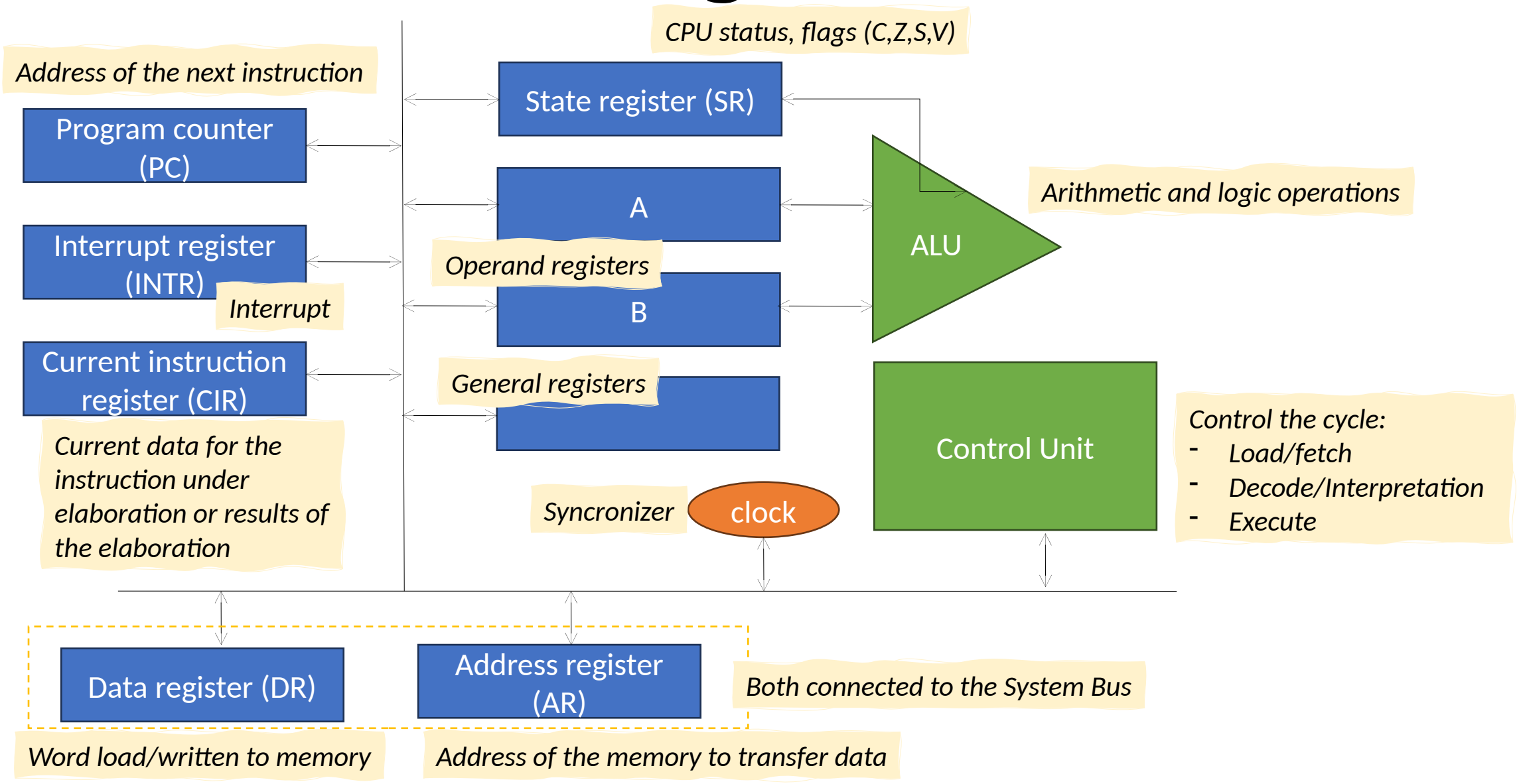
  → *compilation is what makes a program understandable by a pocessor*



Central Processing Unit - CPU (1)

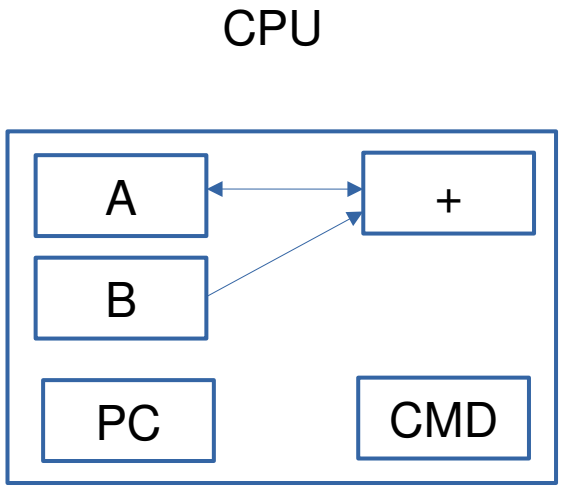


Central Processing Unit - CPU (2)



Simplified Execution Model: CPU

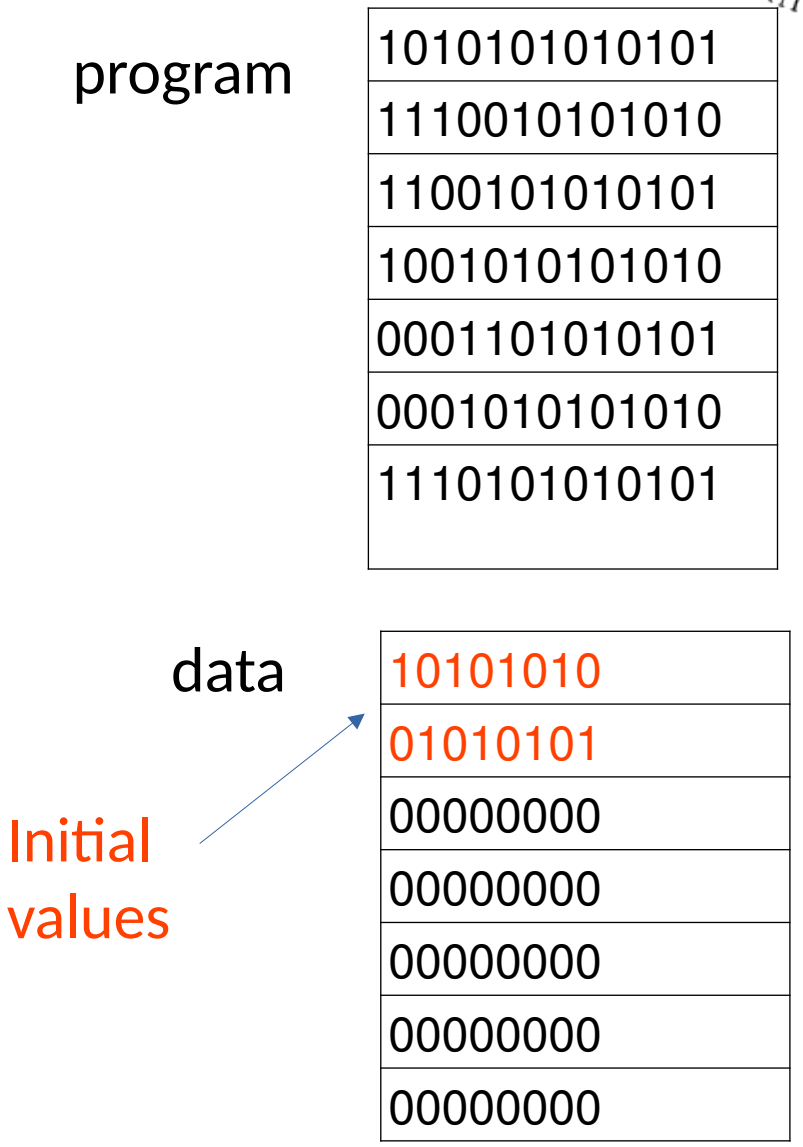
- All operations in a computer are synchronized via a clock
- Every time the clock ticks, an operation can be performed by the devices and the CPU
- Each device can do only one thing at a time
- Faster clock => faster operations
- Clocks moved from KHz to GHz in the last 40 years



Memory (Simplified execution model)

- Programs to be executed reside in primary memory (RAM or ROM)
- The primary memory (RAM) is also used for storing/reading data

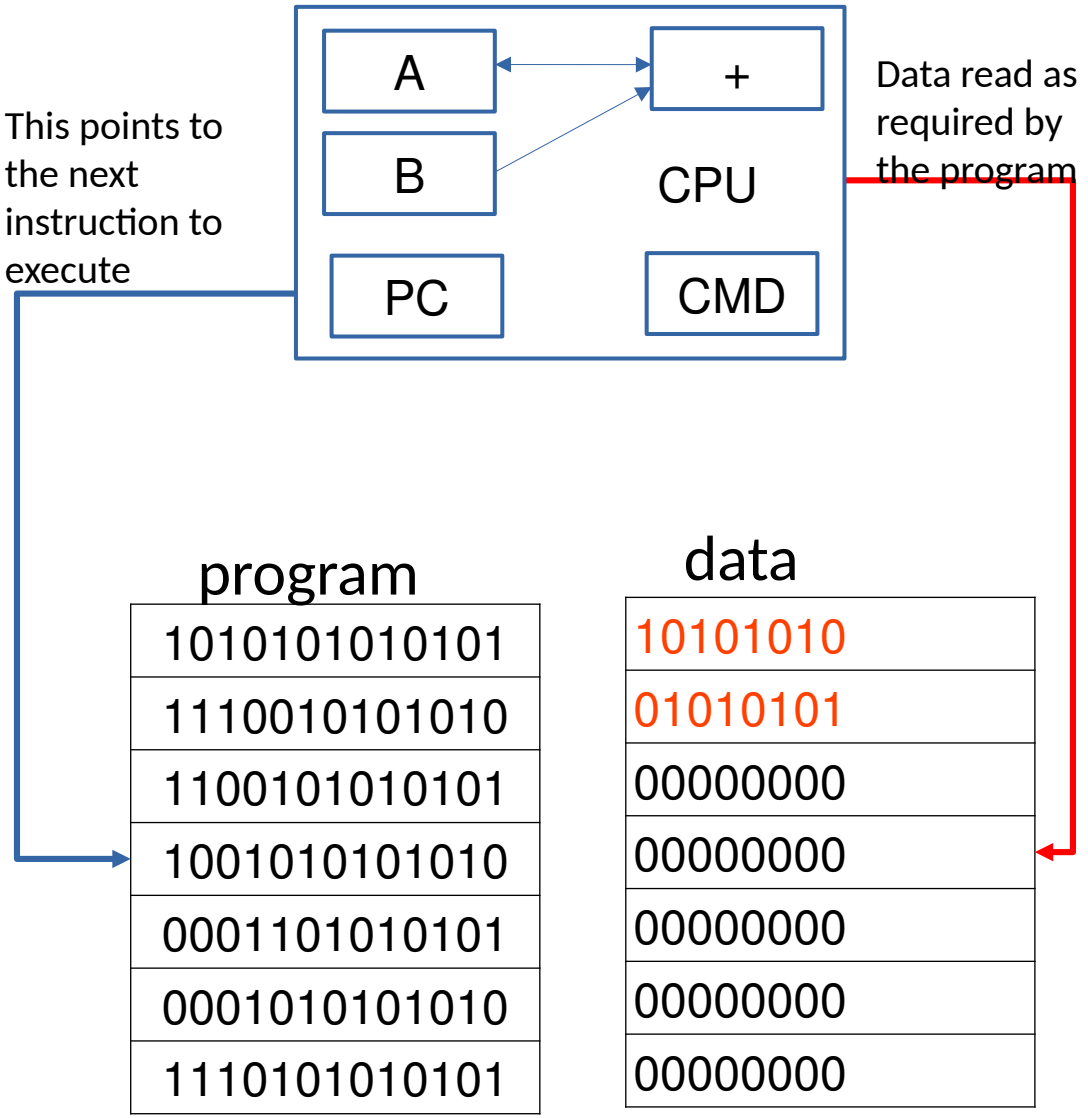
Q. How does a program end up in memory?
The first program to be executed when you boot a computer is stored in the ROM (and the CPU "knows" that).
This software (bootloader) loads other programs (the OS) which make the computer work as we know it.



Execution Model

When the CPU executes a program:

- It sets a PC (program counter) to point to the first instruction of the program to execute
- It reads the instruction pointed by the PC (fetch)
- It executes it, possibly reading and writing data in memory (decode/execute)
- If the instruction requires a “jump” (e.g, in a loop or in
- It increases the PC counter, usually to the next instruction, unless there is an jump to perform (e.g., cycle, condition)

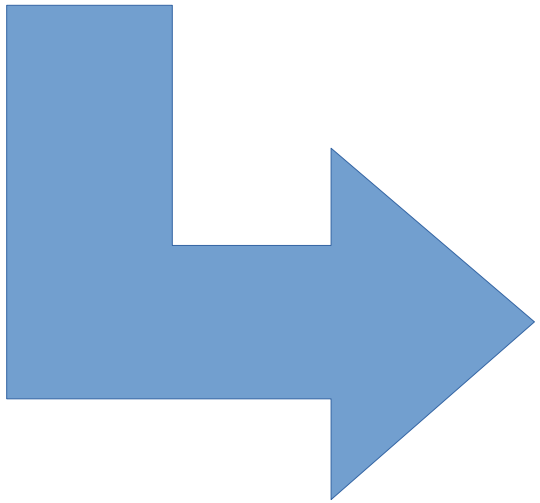


Example

```
int main() {
  int a = 1;
  int b = 2;
  int c = a + b;
}
```

When you compile, the compiler transforms the code in a set of machine-readable instructions (which are stored to disk).

When you execute, the instructions are loaded in memory and a part of the memory is dedicated to holding the data manipulated by the program



program

0x3A	LD A, (0x1000)
0x1000	
0x46	LD B, (1001)
0x1004	
0x80	ADD A, B
0x32	LD (0x1002), A
0x1008	

data

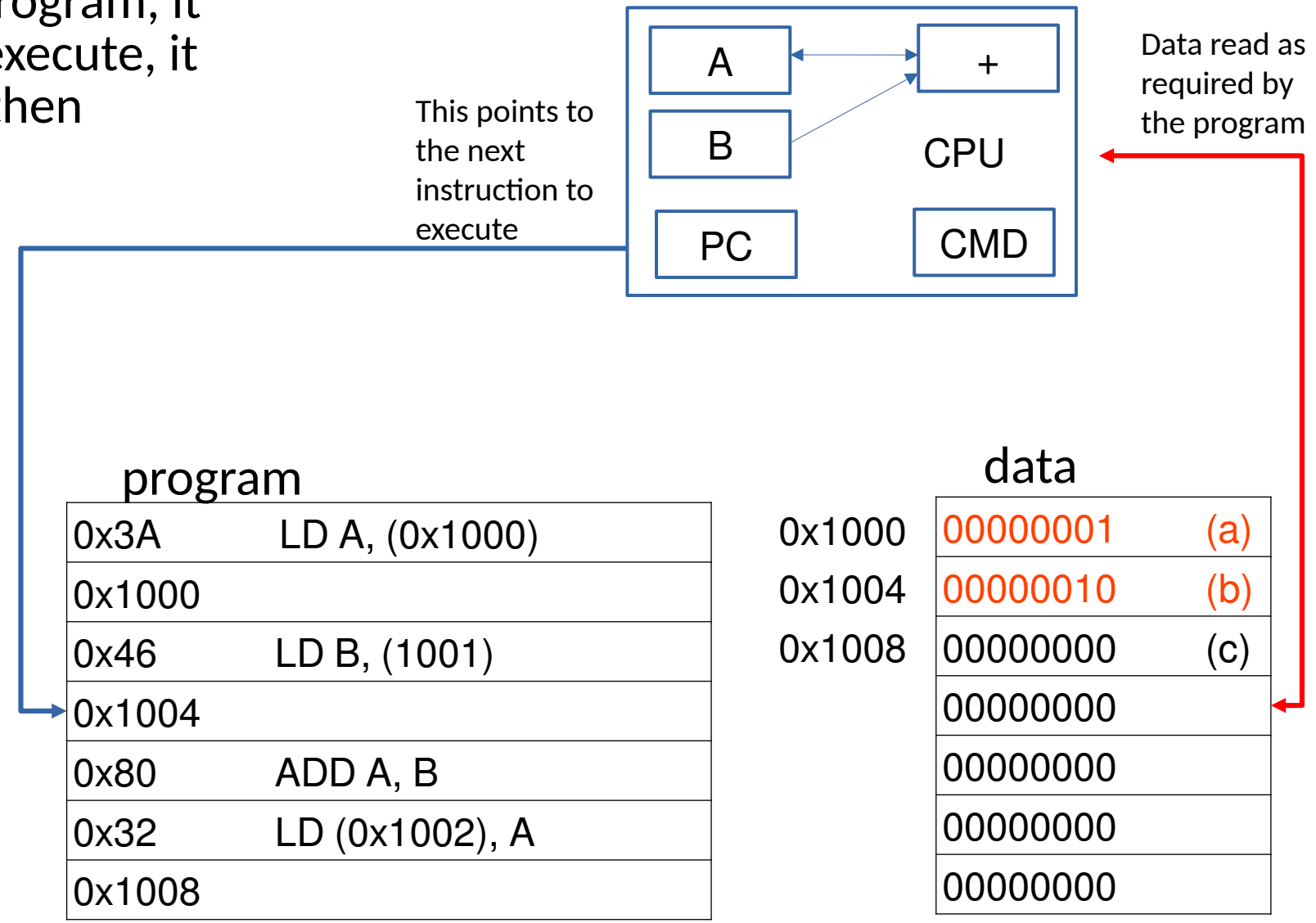
0x1000	00000001	(a)
0x1004	00000010	(b)
0x1008	00000000	(c)
	00000000	
	00000000	
	00000000	
	00000000	

Example

When the CPU executes the program, it **reads** the next instruction to execute, it **decodes** it, it **executes** it and then moves to the next

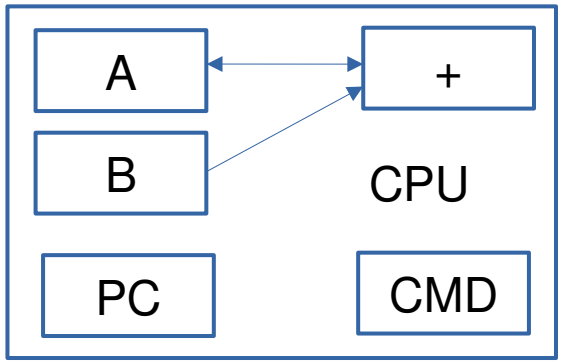
A PC (program counter keeps Track of the next instruction)

Registers are used to hold data transferred by the memory



CPU: how it works – Example (fetch)

- PC starts at 0x00AA
- The address 0x00AA is read: 0x3A
  - is stored in the CPU
- The instruction is decoded:
  - 0x3A means: load next address, read memory at that address and store into register A. The CPU knows and executes
- PC is moved to the next instruction (at 0x00AC)
- We repeat



program

0x00AA	0x3A	LD A, (0x1000)
	0x1000	
0x00AC	0x46	LD B, (1001)
	0x1004	
	0x80	ADD A, B
	0x32	LD (0x1002), A
	0x1008	

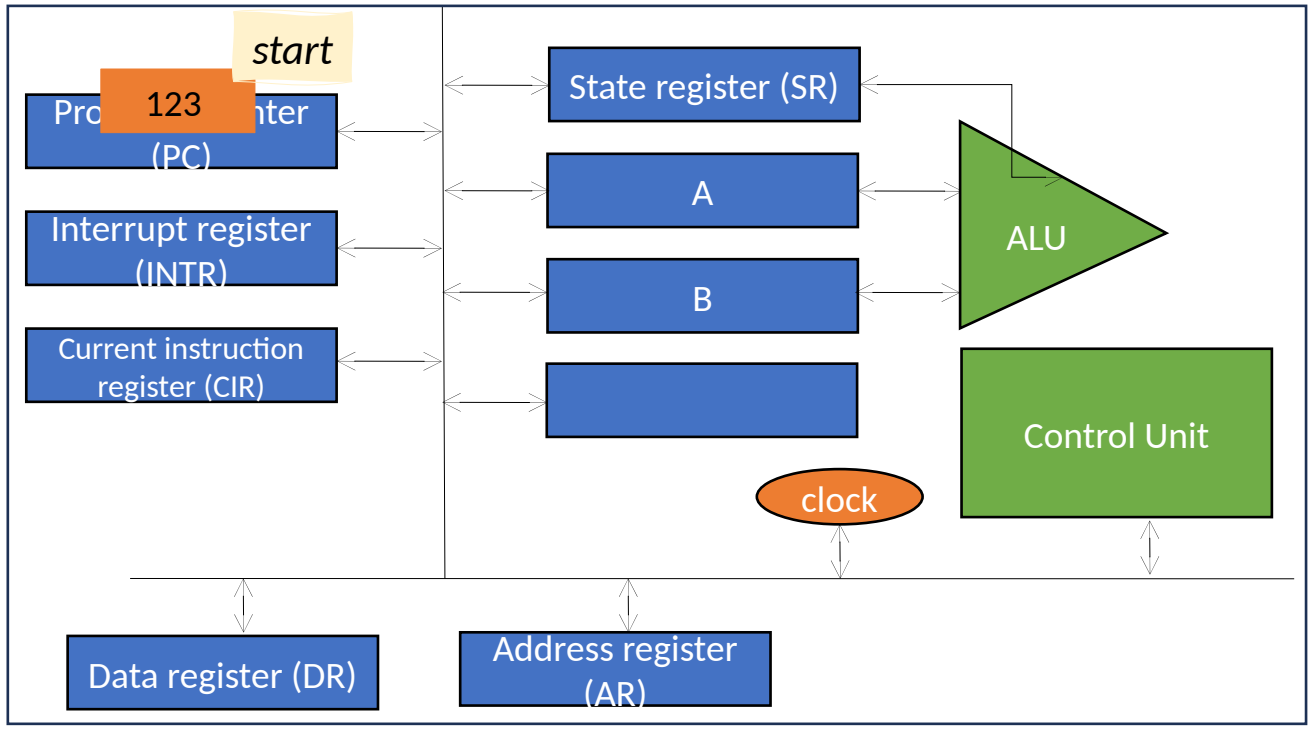
data

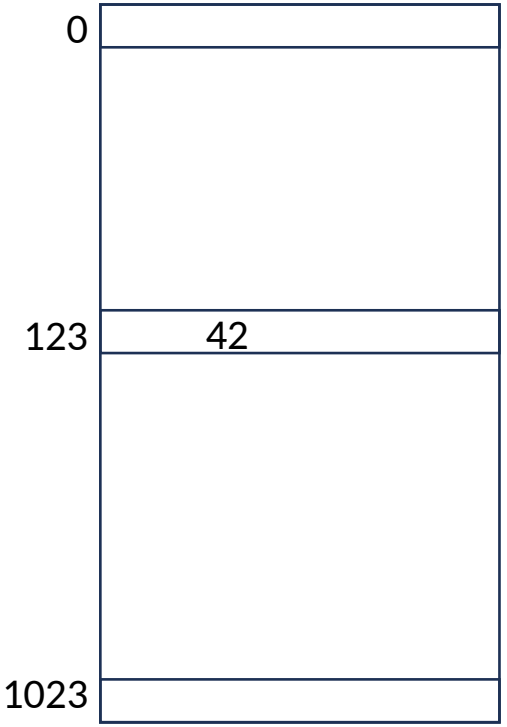
0x1000	00000001	(a)
0x1004	00000010	(b)
0x1008	00000000	(c)
	00000000	
	00000000	
	00000000	
	00000000	

System bus

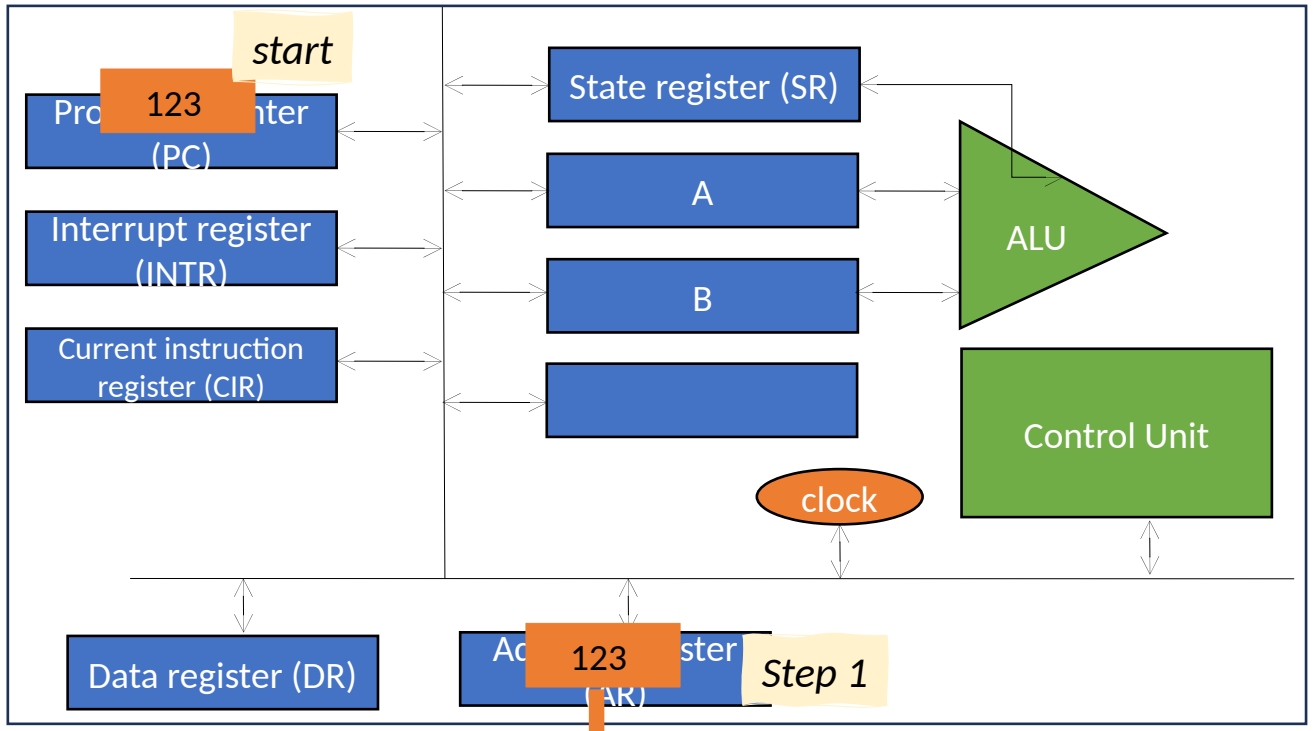
- How does communication between CPU and components happens?
  - The system bus interconnects the various elements of the computer
  - At any time the bus is dedicated to connecting two units, one transmits, and one receives
  - The processor selects the connection to be activated and indicates the operation to be performed (bus mastering)
  - The bus is divided into three sets of lines:
    - Data Bus
    - Address Bus
    - Control lines: carry transfer mode and timing information
  - Example of bus use: writing data from a processor register to memory

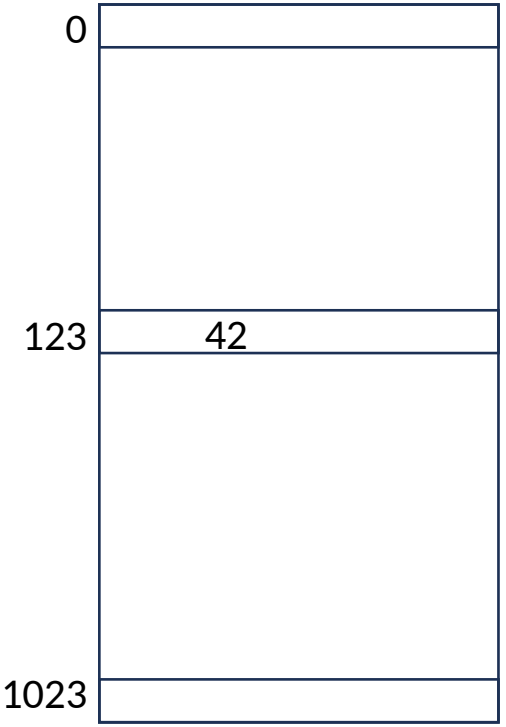
CPU: how it works – fetch phase (4)



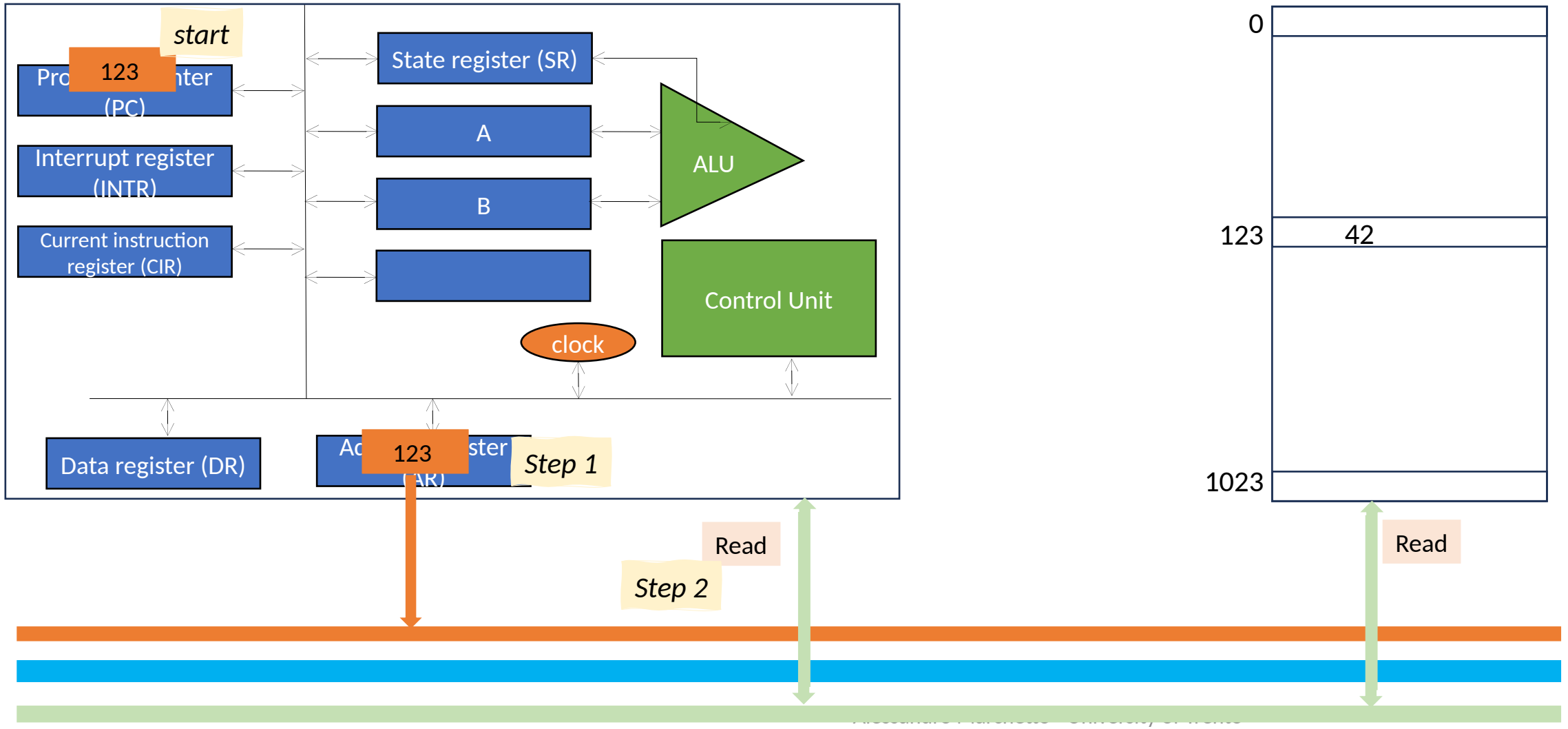


CPU: how it works – fetch phase (5)

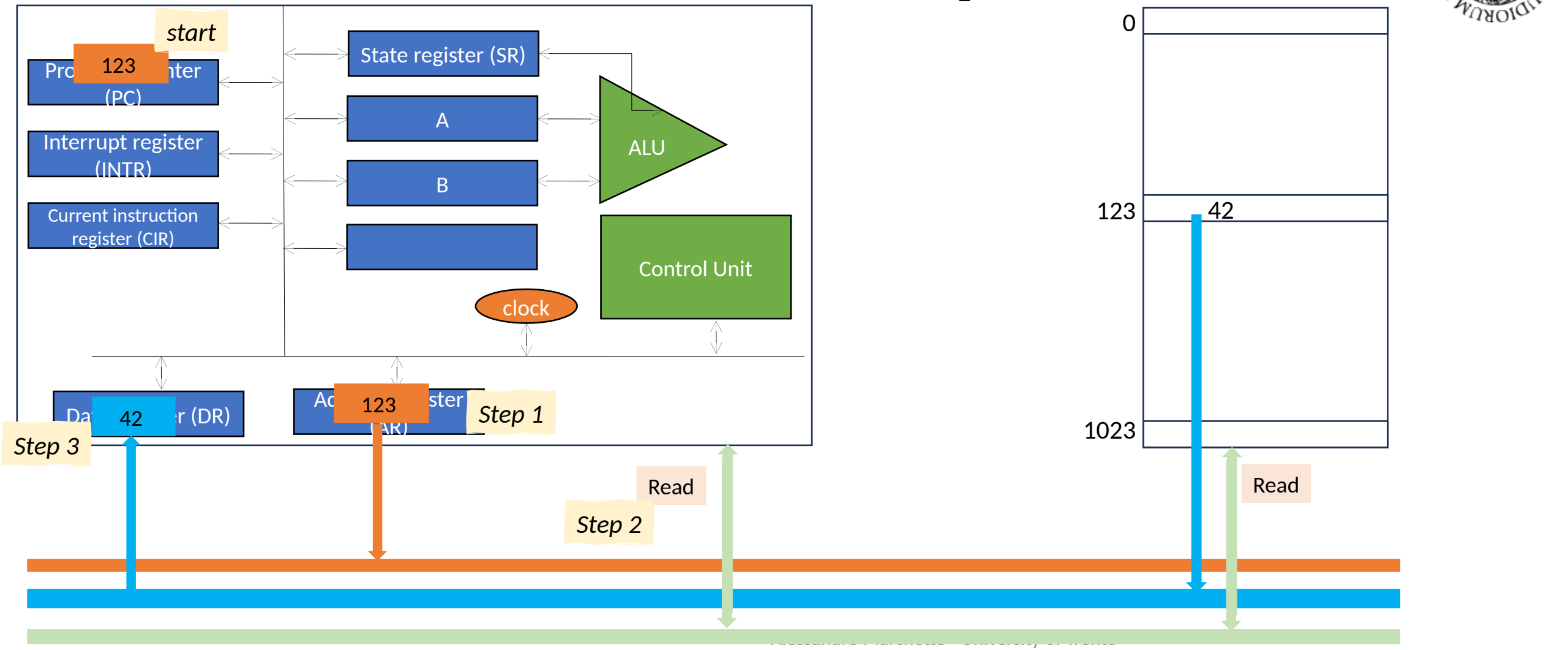




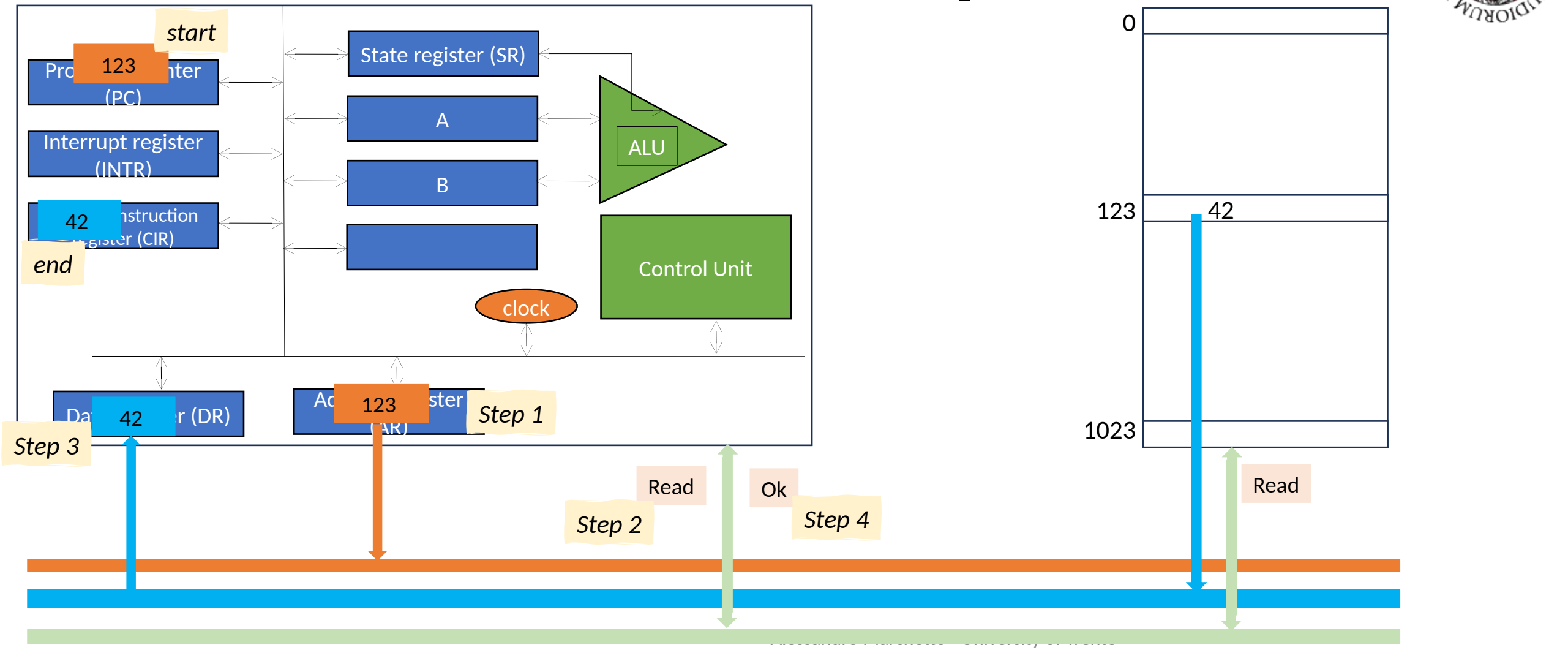
CPU: how it works – fetch phase (6)



CPU: how it works – fetch phase (7)



CPU: how it works – fetch phase (8)



Remark

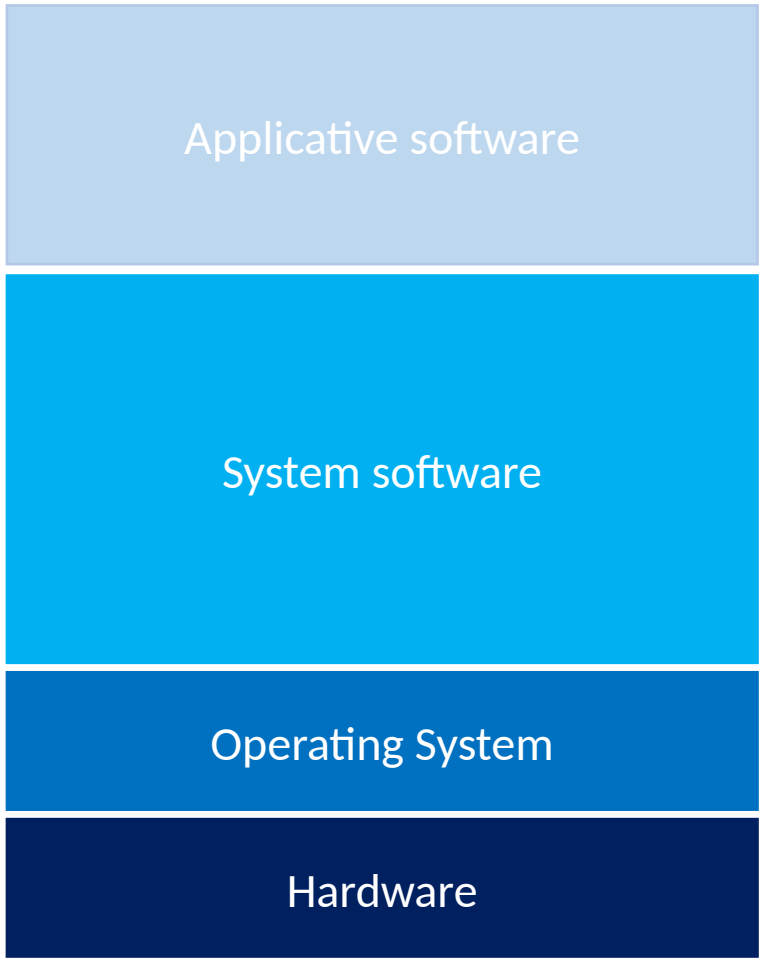
- A high-level programming language abstracts the details of the actual architecture of a computer architecture
  - Variables are an abstraction of memory
  - Instructions are an abstraction of the actual instructions understood by the CPU (one C++ instruction → several machine instructions)

- The execution model, however, is the same:
  - You start from the first instruction
  - Interpret an execute the instruction (updating variable values)
  - Move to the next instruction

More realistic Model

- In general a computer has to execute more than one program at the same time (browser, email, …) and perform different chores (e.g., refresh screen, read keyboard) at the same time
- However, the execution model has a single CPU
- Parallel execution is obtained by “switching context”, that is, executing one instruction of a program, then moving to the next program, …
- Some operations which require fast response (e.g., keyboard) can “interrupt” this sequence and their program take precedence
- **Implementing all of this in hardware is inconvenient. A specialized software (operating systems) becomes responsible of coordinating and orchestrating this behavior**

SW organization in a Computer



Individual Productivity software (e.g., spreadsheets, mail readers)

Programming environment (e.g., Compiler, debugger)

(e.g., Windows, Linux, Mac OS)

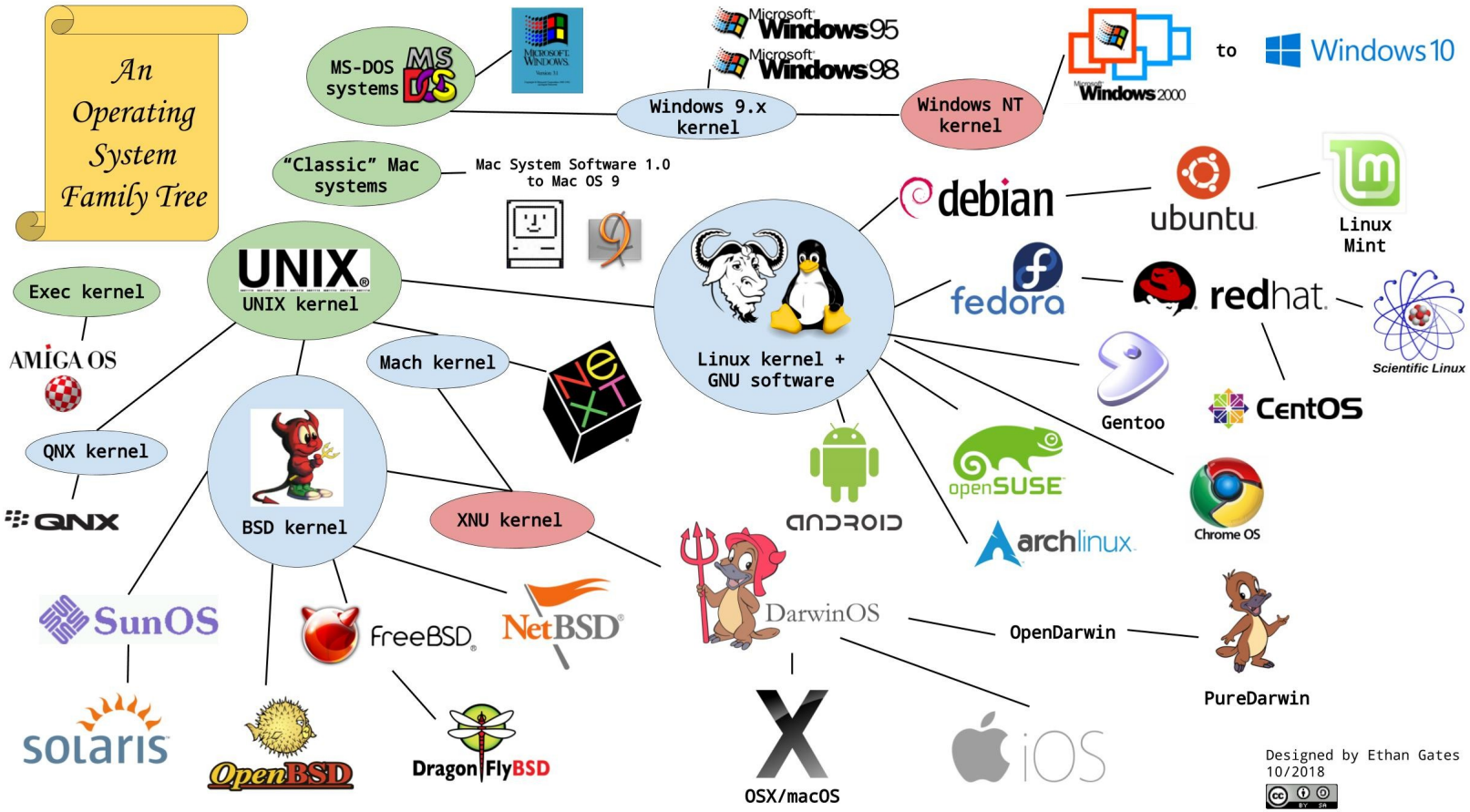
Physical computer's components

Operating system

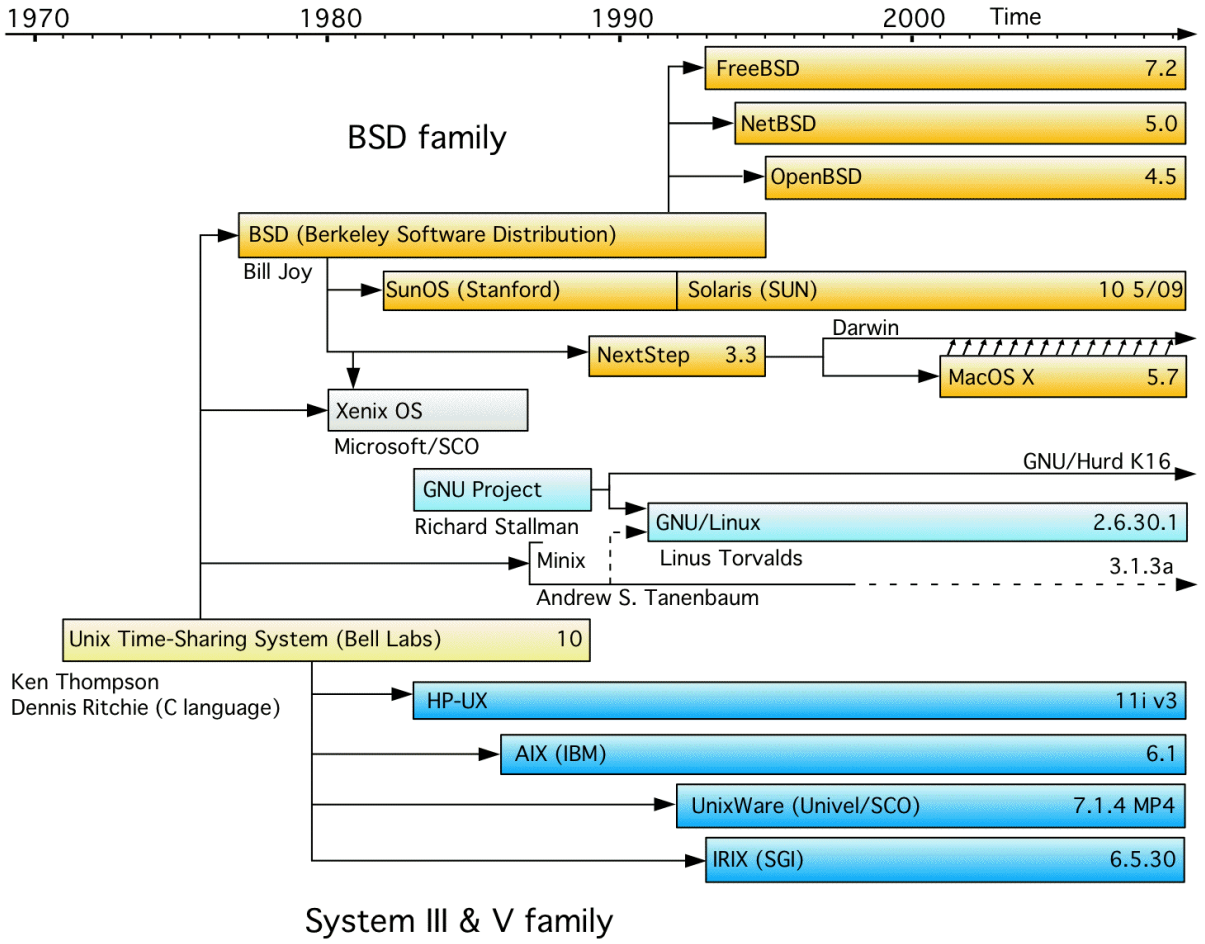
The term “operating system” (OS) refers to the set of programs that operates directly on the physical machine

- It provides to the users with a high-level feature set and by masking its specific characteristics

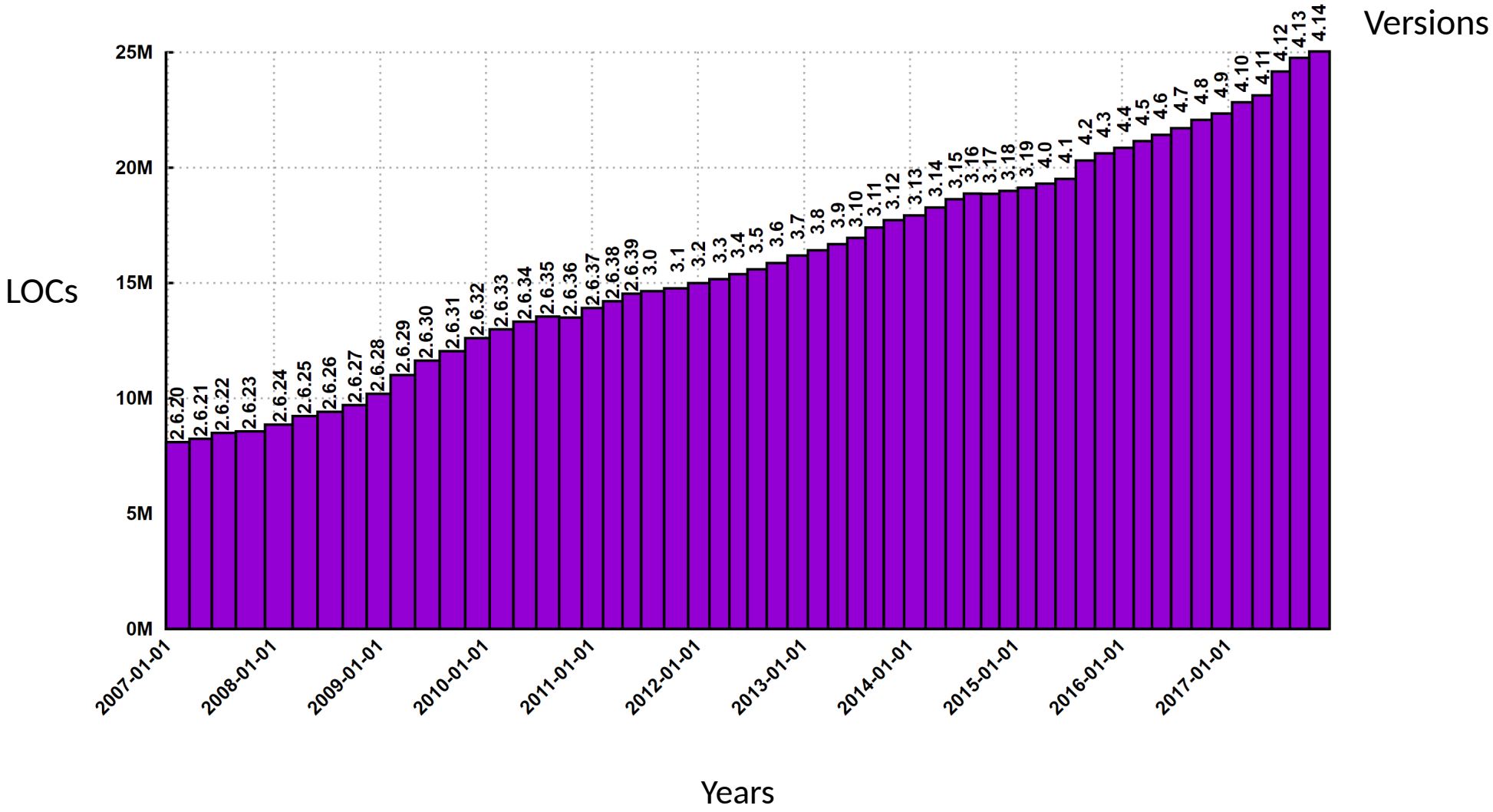
...A bit of History



...A bit of History: focus on Unix



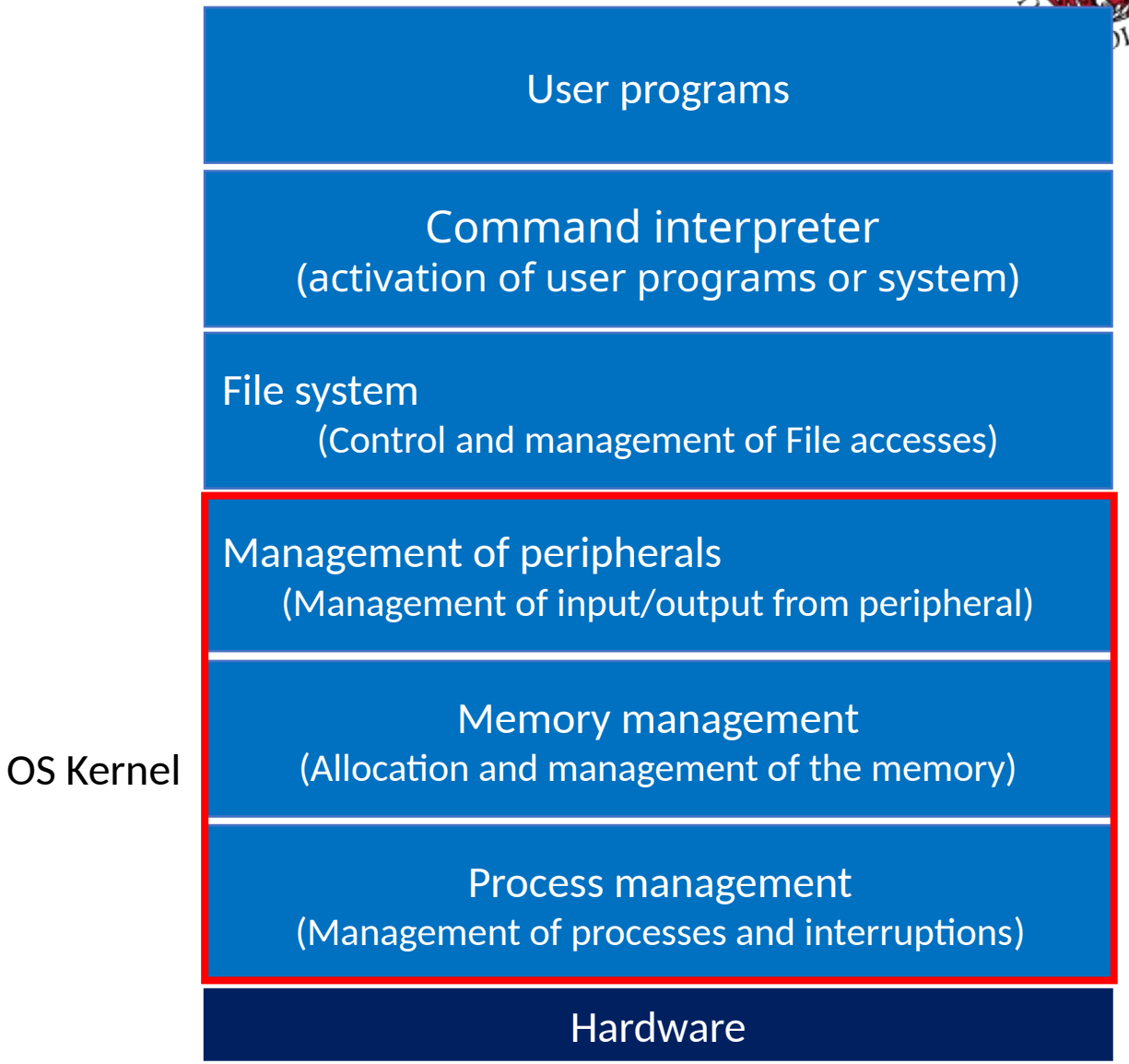
Linux: kernel size



OS architecture

A modern OS is organized according to a "layered" architecture
Each layer provides an abstraction of the layer on which it rests
The architecture allows for a clear separation between interface and implementation of different functionalities

- Each layer consists of a set of programs and libraries



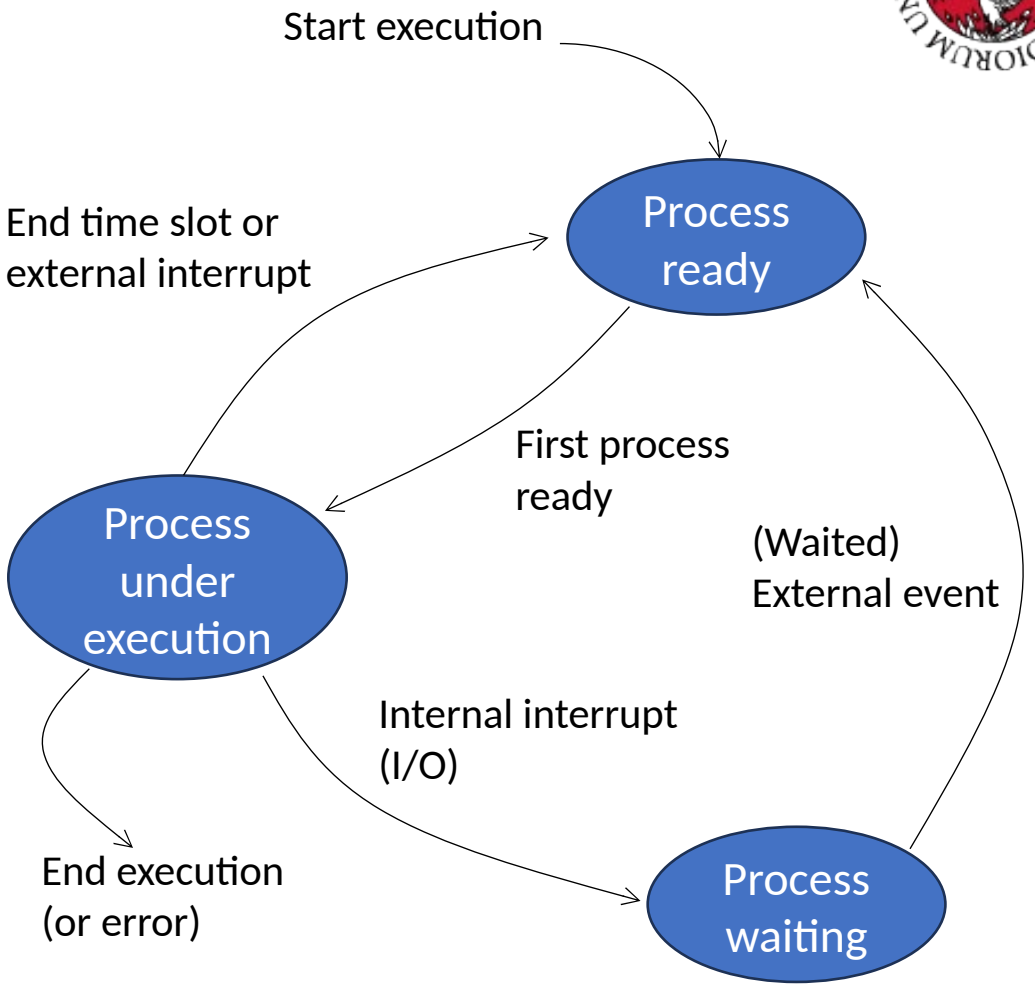
Programs and Processes

While a program is a static entity, a process is a dynamic entity
The process is caused by the running of a program and its execution state (e.g., variable values)

- A process P consists into a pair of elements (E,S), which includes:
  - E: program executable code
  - S: Process state S
- Values contained in central memory
- CPU log values, including program counter (PC)
- The relationship between program and process is not one-to-one
- A program can “generate” multiple processes (concurrent programming)
  - A job is processed by a virtual processor (i.e., it is the process processing unit)

Life cycle of a process

- In a single-processor computer, only one process can run at any given time
  - Other processes are ready or waiting
- A "time interval" to run is assigned to each process, after this interval the processor switches to another process
- It is the so-called time-sharing technique
- It is the OS kernel that decides which processes should run, determining their scheduling
- The typical approach used is named: round-robin ("in turn")
  - The CPU is also released when a process pauses on an I/O request to a device



Memory management

Memory Manager

- It partitions the memory between the various processes that require it
  - It ensures both protection and separation between the different allocated areas
- It manages the memory allocation to processes
  - When requests exceed physical memory limits, it shows processes a virtual memory space

SWAP memory

- The memory manager can use a portion of the secondary memory as a *swap* memory area
  - For example, when a process is suspended, its code and state can be moved to disk to free up central memory.
- This allows processes to be shown virtual memory (much) larger than the central memory
  - E.g., 10% of disk space

…How to see the active processes

Example: command “top” or “ps” for Linux

Example: command “top” for Mac OS

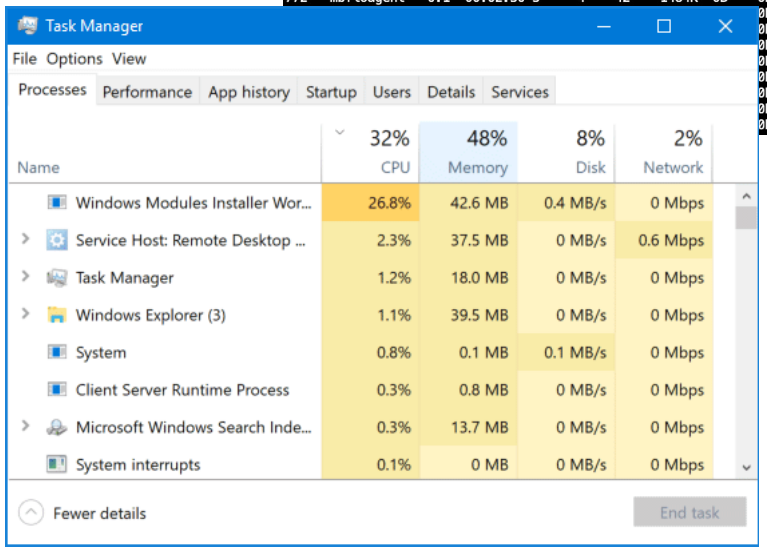
Example: TaskManager in Windows

```
guest-mm2x7k@support21: ~
top - 08:58:11 up 18 min,  1 user,  load average: 0,06, 0,12, 0,14
Tasks: 193 total,   2 running, 142 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0,8 us,  0,1 sy,  0,0 ni, 99,2 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
KiB Mem :  8036980 total,  5598872 free,   831740 used,  1606368 buff/cache
KiB Swap:   998396 total,   998396 free,        0 used.  6784860 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 1237 root      20   0       0      0      0 S   1,3  0,0   0:37.97 Xorg
 2666 guest-m+  20   0 2016192 257036  83708 S   1,3  3,2   0:50.88 compiz
 4148 guest-m+  20   0  593456  35748  28420 S   1,0  0,4   0:02.50 gnome-term+
 2624 guest-m+  20   0  868668  68984  54808 S   0,3  0,9   0:04.23 nautilus
    1 root      20   0       0      0      0 S   0,0  0,0   0:01.25 systemd
    2 root      20   0       0      0      0 S   0,0  0,0   0:00.00 kthreadd
    4 root       0 -20       0      0      0 I   0,0  0,0   0:00.00 kworker/0:+
    5 root      20   0       0      0      0 I   0,0  0,0   0:00.01 kworker/u8+
    6 root       0 -20       0      0      0 I   0,0  0,0   0:00.00 mm_percpu_+
    7 root      20   0       0      0      0 S   0,0  0,0   0:00.02 ksoftirqd/0
    8 root      20   0       0      0      0 I   0,0  0,0   0:01.68 rcu_sched
    9 root      20   0       0      0      0 I   0,0  0,0   0:00.00 rcu_bh
   10 root      rt   0       0      0      0 S   0,0  0,0   0:00.00 migration/0
   11 root      rt   0       0      0      0 S   0,0  0,0   0:00.00 watchdog/0
   12 root      20   0       0      0      0 S   0,0  0,0   0:00.00 cpuhp/0
   13 root      20   0       0      0      0 S   0,0  0,0   0:00.00 cpuhp/1
   14 root      rt   0       0      0      0 S   0,0  0,0   0:00.00 watchdog/1
```

```
ofershmueli — top — 132×39
Processes: 430 total, 2 running, 428 sleeping, 1830 threads                                                 13:52:03
Load Avg: 2.15, 2.44, 3.61  CPU usage: 5.98% user, 3.35% sys, 90.65% idle  SharedLibs: 400M resident, 98M data, 112M linkedit.
MemRegions: 103565 total, 5119M resident, 248M private, 2000M shared. PhysMem: 16G used (1880M wired), 282M unused.
VM: 2002G vsize, 1993M framework vsize, 0(0) swapins, 0(0) swapouts. Networks: packets: 268613/320M in, 161931/27M out.
Disks: 1283605/13G read, 264305/3839M written.

PID   COMMAND      %CPU  TIME     #TH   #WQ  #PORT MEM    PURG  CMPR PGRP PPID STATE    BOOSTS          %CPU_ME %CPU_OTHRS UID
1547  Terminal     34.9  00:02.44 8     3    260-  88M+   13M-  0B   1547 1    sleeping *0[11]          0.01040 0.00000    501
314   WindowServer 11.9  02:17.84 11    4    9684  386M+  10M-  0B   314  1    sleeping *0[1]           1.04851 0.00057    88
5884  screencaptur 4.4   00:00.44 2     1    54    3168K+ 620K  0B   1549 1549 sleeping *0[1]           0.12397 0.00000    501
0     kernel_task  4.2   04:44.86 186/8 0    0     79M+   0B    0B   0    0    running   0[0]           0.00000 0.00000    0
5883  top          3.6   00:00.65 1/1   0    27    5112K  0B    0B   5883 5863 running   *0[1]          0.00000 0.00000    0
507   AppleUserHID 1.4   00:04.71 3     2    67    860K   0B    0B   507  1    sleeping  0[1]           0.00000 0.00000    270
5540  Google Chrom 1.0   03:06.12 18    1    280   225M   4096B 0B   1543 1543 sleeping *0[9]           0.00000 0.00000    501
511   AppleUserHID 0.9   00:03.31 2     1    67    1108K  0B    0B   511  1    sleeping  0[1]           0.00000 0.00000    270
1659  Creative Clo 0.8   00:14.91 28    1    365   38M    8192B 0B   1659 1    sleeping *0[1]           0.00000 0.00000    501
217   hidd         0.7   00:06.04 6     3    232   4688K  0B    0B   217  1    sleeping *0[1]           0.02191 0.04167    261
229   tccd         0.6   00:02.61 3     2    53+   8456K+ 36K   0B   229  1    sleeping *0[383+]        0.00000 0.67699    0
1622  Google Chrom 0.5   00:15.62 16    1    141   122M-  4096B 0B   1543 1543 sleeping *0[6]           0.00000 0.00000    501
5885  screencaptur 0.4   00:00.09 3     1    153   4096K+ 0B    0B   5885 1    sleeping *0[164+]        0.00000 0.12397    501
257   trustd       0.3   00:30.50 3     2    97    12M+   7048K 0B   257  1    sleeping *0[2424+]       0.00000 0.31558    0
360   nessusd      0.1   00:11.60 25    0    41    33M    0B    0B   169  169  sleeping *0[1]           0.00000 0.00000    0
1683  AdobeResourc 0.1   00:03.94 12    1    112   7988K  0B    0B   1683 1    sleeping *1[19]          0.00000 0.00000    501
1543  Google Chrom 0.1   02:04.48 31    1    722   171M   4520K 0B   1543 1    sleeping *0[283]         0.00000 0.00000    501
1648  TeamViewer_D 0.1   00:02.60 4     1    89    5676K  0B    0B   1648 1    sleeping *0[1]           0.04460 0.00000    501
155   fseventsd    0.1   00:06.11 10    1    240   7384K  0B    0B   155  1    sleeping *0[1]           0.00000 0.00000    0
1660  BlueJeansHel 0.1   00:00.39 4     3    42+   3204K+ 0B    0B   1660 1    sleeping *0[1]           0.00000 0.00000    501
1843  AdobeCRDaemo 0.1   00:01.35 4     3    57    1716K  0B    0B   1843 1826 sleeping *0[8]           0.02167 0.00000    501
1603  Google Chrom 0.1   05:06.71 19    1    272   210M   4096B 0B   1543 1543 sleeping *0[9]           0.00000 0.00000    501
1     launchd      0.1   00:12.94 2     1    3287  21M    0B    0B   1    0    sleeping  0[0]           0.00000 0.07920    0
772   mbfloagent   0.1   00:02.30 5     4    42    1484K  0B    0B   772  1    sleeping  0[32]          0.00000 0.02570    501
                                                                                0B   2165 2151 sleeping *0[1]           0.01206 0.00000    501
                                                                                0B   160  1    sleeping *0[1]           0.00000 0.04460    0
                                                                                0B   178  1    sleeping *0[1]           0.02613 0.00000    0
                                                                                0B   189  1    sleeping *0[1]           0.00000 0.03588    0
                                                                                0B   1659 1659 sleeping *0[1]           0.00000 0.00000    501
                                                                                0B   1749 1659 sleeping *0[1]           0.01233 0.00000    501
                                                                                0B   1543 1543 sleeping *0[4]           0.00000 0.00000    501
                                                                                0B   1543 1543 sleeping *0[6]           0.00000 0.00000    501
```



Device manager

It provides the upper layers with a set of high-level procedures for accessing different peripherals
It gives to every process the visibility of a set of dedicated virtual peripherals

- It manages, at least in part, the peripheral malfunctions

Device Manager Components

- Physical drivers
  - Manage the device directly, polling or interrupting
  - They are part of the OS kernel of the operating system
- Logical drivers
  - They operate above the core by providing the implementation of peripheral access primitives
- Special files
  - Mask devices by providing them with file-like access
- Spooling system
  - Managing access to printers and other devices based on a queue concept

Command Interpreter

The command interpreter is responsible for managing commands coming directly from the user
It represents the interface with the user
In modern OS, most of the features are realized through a suitable graphical interface, based on windows and icons

- However, the possibility/necessity of using the text command interpreter remains

...File System

File System

Each file in the (secondary) memory is associated to a descriptor

The file descriptor contains characteristic information of the file such as: name, size, start location on disk, owner, access permissions, ...

Each file is contained in a folder represented, by the OS, as a set of descriptors

The folder may contain other folders in a tree whose leaves are files

- Each file is uniquely identified by its pathname
  - e.g., c:\alpha\beta\gamma.txt

File System Manager

- Is responsible for the management of peripherical devices
- Provides the upper layers with a set of procedures for accessing the file system
  - uniquely identify a file, read and write data, manage concurrent access to a file, ...
  - masks the characteristics of the device on which the file resides
  - Ensures protection in file access